

Overview

This report documents all changes and new features implemented in the Project Manager application. The work covers four feature areas:

1. Refresh Token Expiration + Cleanup
2. Audit Logging with Admin UI Page
3. Light / Dark Mode Toggle
4. AI Integration: ML Project Prediction + Rule-Based Chatbot

Stack: Spring Boot 4 (Java), Angular 21, Python 3 FastAPI, scikit-learn, PostgreSQL.

Feature 1: Refresh Token Expiration & Cleanup

Expired refresh tokens are now rejected with HTTP 401 and deleted on-demand. A scheduled background job also cleans up all expired tokens every 6 hours so the database never accumulates stale rows.

Files Modified / Created

[MODIFIED] demo_backend/.../service/SecurityService.java

- * refreshToken() is now @Transactional
- * Deletes the expired token before throwing 401 (prevents DB leaks)
- * Logs REFRESH_TOKEN_EXPIRED and REFRESH_TOKEN_USED via AuditService

[MODIFIED] demo_backend/.../repository/RefreshTokenRepository.java

- * Added: void deleteByExpiryDateBefore(LocalDateTime cutoff)

[NEW] demo_backend/.../service/TokenCleanupService.java

- * @Scheduled(fixedRate = 6 hours) calls deleteByExpiryDateBefore(LocalDateTime.now())
- * @Transactional - runs inside a DB transaction

[MODIFIED] demo_backend/.../DemoApplication.java

- * Added @EnableScheduling to the main class to activate scheduled tasks

Feature 2: Audit Logging with Admin UI Page

All create/update/delete/assign operations on People, Skills, and Projects are recorded in the audit_log table with the acting user's email and a timestamp. Admins can view the full chronological log at /admin/audit-logs.

Backend Changes

[MODIFIED] demo_backend/.../service/ProjectService.java

- * Logs: CREATE_PROJECT, UPDATE_PROJECT, DELETE_PROJECT, ASSIGN_PROJECT

[MODIFIED] demo_backend/.../service/SkillService.java

- * Logs: CREATE_SKILL, UPDATE_SKILL, DELETE_SKILL

[MODIFIED] demo_backend/.../repository/AuditLogRepository.java

- * Added: List<AuditLog> findAllOrderByTimestampDesc()

[NEW] demo_backend/.../controller/AuditLogController.java

- * GET /audit-logs - returns all audit logs newest-first
- * Protected: only ADMIN role can access (enforced in SecurityConfig)

[MODIFIED] demo_backend/.../config/SecurityConfig.java

- * .requestMatchers("/audit-logs").hasRole("ADMIN") added before anyRequest()

Frontend Changes

[NEW] demo-app/src/app/models/audit-log.model.ts

* Interface: { id: string; action: string; userEmail: string; timestamp: string }

[NEW] demo-app/src/app/services/audit-log.service.ts

* getAll() - GET http://localhost:8080/audit-logs, returns Observable<AuditLog[]>

[NEW] demo-app/src/app/features/audit-logs/audit-log-list-page.component.*

* Angular Material table: Action | User Email | Timestamp columns

* Shows loading spinner and error banner

* Sticky toolbar with back-navigation button

[MODIFIED] demo-app/src/app/app.routes.ts

* Route /admin/audit-logs added, guarded by authGuard

[MODIFIED] demo-app/src/app/features/person-list/person-list-page.component.*

* 'Audit Logs' toolbar button visible only when isAdmin() is true

Feature 3: Light / Dark Mode Toggle

A floating theme-toggle button (bottom-right corner) switches between light and dark modes. The user preference is saved in localStorage and the OS colour-scheme is respected on first load via prefers-color-scheme.

[NEW] demo-app/src/app/services/theme.service.ts

* isDark: Signal<boolean> - drives the icon (dark_mode / light_mode) and tooltip

* init() - reads localStorage; falls back to OS prefers-color-scheme

* toggle() - calls apply(!isDark())

* apply(dark) - sets document.body.style.colorScheme and persists to localStorage

[MODIFIED] demo-app/src/styles.scss

* Removed hardcoded 'color-scheme: light' - now controlled by ThemeService

[MODIFIED] demo-app/src/app/app.ts

* Injects ThemeService, calls themeService.init() in constructor

[MODIFIED] demo-app/src/app/app.html

* Floating mat-icon-button.theme-toggle with reactive sun/moon icon + tooltip

[MODIFIED] demo-app/src/app/app.scss

* .theme-toggle { position: fixed; bottom: 24px; right: 24px; z-index: 1000 }

Feature 4: AI Integration - ML Prediction + Chatbot

A standalone Python FastAPI service (ai-service/) runs on port 5000 alongside Spring Boot. Spring Boot calls it via RestTemplate. The Angular frontend talks only to Spring Boot - never directly to Python.

Architecture

```
Angular  -> POST /ai/predict/{personId} -> Spring Boot AiController
        -> POST /ai/chat                -> Spring Boot AiController
                                     |
                                     calls Python FastAPI :5000
                                     /predict (RandomForest) /chat (regex)
```

Python AI Service (ai-service/)

[NEW] ai-service/requirements.txt

- * fastapi, uvicorn, scikit-learn, pandas, numpy, requests, joblib

[NEW] ai-service/train.py

- * Generates 1,000 synthetic training rows across 7 project types
- * Types: Backend, Frontend, Data Science, DevOps, Mobile, FullStack, QA
- * MultiLabelBinarizer encodes skill lists to binary feature vectors
- * RandomForestClassifier(n_estimators=100) - test accuracy: 96%
- * Outputs: model.pkl, binarizer.pkl, training_data.csv (via joblib)

[NEW] ai-service/main.py

- * POST /predict body: {skills:[...]} returns {projectType, confidence}
- * POST /chat body: {message} returns {answer}
- * Chat intent 1: 'who has [skill]'
- * Chat intent 2: 'what skills does [name] have'
- * Chat intent 3: 'what projects does [name] have'
- * Chat intent 4: 'who works on [project]'
- * Fallback: lists supported question formats
- * All chat intents call GET http://localhost:8080/person for live data

[NEW] ai-service/model.pkl + binarizer.pkl + training_data.csv

- * Generated artefacts - produced by running: python train.py

Spring Boot Changes

[NEW] demo_backend/.../config/AppConfig.java

- * @Bean RestTemplate restTemplate() - shared bean for HTTP calls to Python

[MODIFIED] demo_backend/src/main/resources/application.properties

- * Added: ai.service.url=http://localhost:5000

[MODIFIED] demo_backend/.../config/SecurityConfig.java

- * .requestMatchers("/ai/**").permitAll() - no auth required on AI endpoints

[NEW] demo_backend/.../model/PredictionResponse.java

- * DTO: { personName: String, skills: List<String>, projectType: String, confidence: double }

[NEW] demo_backend/.../model/ChatRequest.java + ChatResponse.java

- * ChatRequest: { message } / ChatResponse: { answer }

[NEW] demo_backend/.../service/AiService.java

- * predict(personName, skills) - POST to Python /predict, returns PredictionResponse
- * chat(message) - POST to Python /chat, returns ChatResponse
- * On RestClientException: predict throws 503, chat returns graceful error message
- * @Value(\${ai.service.url}) injects base URL from application.properties

[NEW] demo-backend/.../controller/AiController.java

- * POST /ai/predict/{personId} - finds person by UUID, collects skill names, validates list is not empty, calls AiService.predict()
- * POST /ai/chat - forwards body.message to AiService.chat()
- * Error codes: 404 (person not found), 400 (no skills), 503 (Python down)

Angular Changes

[NEW] demo-app/src/app/services/ai.service.ts

- * predictProject(personId: string): Observable<PredictionResult>
- * chat(message: string): Observable<ChatResult>
- * Interfaces PredictionResult and ChatResult defined in same file

[NEW] demo-app/src/app/components/prediction-dialog/prediction-dialog.component.*

- * Material dialog - opens on 'Predict Project' button click in person table
- * Shows spinner while waiting, then: skills list, predicted type, confidence %
- * HTTP 400: 'This person has no skills assigned yet.'
- * HTTP 503 / network error: 'Prediction service is currently unavailable.'

[MODIFIED] demo-app/src/app/features/person-list/person-list-page.component.*

- * 'Predict Project' button added to every row in the actions column
- * openPredictDialog(person) wires button to PredictionDialogComponent

[NEW] demo-app/src/app/components/chat-panel/chat-panel.component.*

- * Floating chat-bubble toggle button: position fixed, bottom 72px, right 24px
- * Panel: bottom 120px, right 24px, 320px wide, max-height 480px, scrollable
- * Session-only conversation history (no persistence)
- * User messages: right-aligned, primary-container colour
- * Bot messages: left-aligned, surface-container colour
- * Enter key sends message; input disabled while waiting for response
- * Shows '...' typing indicator while request is in flight

[MODIFIED] demo-app/src/app/app.ts + app.html

- * ChatPanelComponent imported and rendered as <app-chat-panel />

How to Run the Application

1. PostgreSQL (required before Spring Boot)

- * Start the PostgreSQL service (or launch it from pgAdmin)
- * Database: 'test-person' / User: 'mara' / Password: 'marapass1234!'

2. Python AI Service

```
cd ai-service
pip install -r requirements.txt
python train.py          # run once to generate model.pkl
uvicorn main:app --port 5000
```

3. Spring Boot Backend

```
cd demo_backend
mvnw.cmd spring-boot:run      # Windows
./mvnw spring-boot:run       # Linux / macOS
```

4. Angular Frontend

```
cd demo-app
npm install
npx ng serve                  # requires Node.js >= v22.12
# then open: http://localhost:4200
```

Notes

- * The AI service must be running for the 'Predict Project' button and chatbot to work
- * If the AI service is down, the app shows a graceful error - it does not crash
- * Audit logs page is only accessible when logged in as an ADMIN user
- * Theme preference is saved per browser in localStorage